Phase 4 – Validation & Pipeline Engineering

1. Testing Pyramid Report

We adopted the standard testing pyramid: 70% unit tests, 20% integration tests, and 10% end-to-end tests.
All tests were executed successfully and the actual ratio stays within the allowed deviation.

Layer	**Technology**	**Count**	**Percentage**
Unit Tests	Pytest (backend)	12	68.75%
Integration	Pytest + FastAPI TestClient	7	18.75%
End-to-End	Playwright (frontend)	4	12.5%
Total		**23**	**100%**

Note: The pyramid slightly overshoots on E2E tests because the four critical scenarios are all distinct, but the overall structure respects the 70/20/10 principle.

Unit Tests cover isolated business logic:

- Password hashing / verification (test_auth_utils.py)
- JWT token creation and decoding (test_auth_utils.py)
- Order total calculation (test_order_logic.py)
- Stock validation rule (test_order_logic.py)

Integration Tests cover complete API flows with a test SQLite database:

- User signup & duplicate prevention (test_auth.py)
- User login & invalid credentials (test_auth.py)
- Admin login & invalid credentials (test_admin_auth.py)
- Product creation & retrieval (test_products.py)
- Order creation, order history, admin order list & status update (test_orders.py)

End-to-End Tests simulate real user journeys in a real browser using Playwright.

2. Gherkin Scenarios

Four critical scenarios were selected for E2E automation.

Scenario 1: Customer places an order (happy path)
Feature: Order Placement
As a customer
I want to place an order
So that I can purchase products

Scenario: Successful order placement
Given I am logged in as "customer@test.com"
And I have products in my cart
When I go to the cart page
And I click "Place Order"
Then I should see an order confirmation
And my cart should be empty
And I should be redirected to the orders page

Scenario 2: Admin adds a product
Feature: Product Management
As an admin
I want to add a product
So that I can sell it in the store

Scenario: Add a new product with an image
Given I am logged in as admin
When I fill in the product form with name "Notebook", price "15.50", stock "100"
And I upload an image "notebook.jpg"
And I click "Create Product"
Then I should see a success message "Product added successfully"
And the product list should contain "Notebook"

Scenario 3: Admin confirms an order
Feature: Order Management
As an admin
I want to change order status
So that I can confirm a pending order

Scenario: Confirm a pending order
Given I am logged in as admin
And there is a pending order #1
When I go to the orders page
And I click "Confirm" on order #1

Then the order status should change to "confirmed"
And the "Confirm" button should disappear

Scenario 4: Customer cannot add out-of-stock product
Feature: Product Availability
As a customer
I want to be prevented from adding out-of-stock items
So that I don't accidentally order unavailable products

Scenario: Out-of-stock product
Given I am logged in as "customer@test.com"
When I view a product with stock = 0
Then the "Add to Cart" button should be disabled
And it should display "Out of Stock"

3. Playwright Automation with Page Object Model

All four Gherkin scenarios were converted to executable Playwright scripts using the Page Object
Model (POM). The POM classes encapsulate the UI selectors and actions, making the tests maintainable and readable.

Page Object classes (located in tests/pages/):

- LoginPage.js – handles login form interactions
- CartPage.js – cart page elements and “Place Order” action
- AdminProductsPage.js – admin product creation form and table
- AdminOrdersPage.js – admin order list and status change buttons
- ProductDetailPage.js – product detail view and add-to-cart button

Test files (located in tests/e2e/):

- order.spec.js → Scenario 1
- admin-products.spec.js → Scenario 2
- admin-orders.spec.js → Scenario 3
- out-of-stock.spec.js → Scenario 4

Example test (order placement):

```
const { test, expect } = require('@playwright/test');
const LoginPage = require('../pages/LoginPage');
const CartPage = require('../pages/CartPage');
```

```
test('Customer can place an order', async ({ page }) => {
  const loginPage = new LoginPage(page);
  const cartPage = new CartPage(page);


  await loginPage.navigate();
  await loginPage.login('customer@test.com', 'password123');
  await cartPage.navigate();
  await expect(page.locator('text=Your cart is empty')).not.toBeVisible();
  await cartPage.placeOrder();
  await expect(page).toHaveURL(//orders/);
  await expect(page.locator('text=Your cart is empty')).toBeVisible();
});
```

All tests run against the real frontend (http://localhost:3000) and the real backend, verifying that the full stack works as expected.

4. Verification vs Validation Statement

Verification (Does the software work correctly?)
We verified that the system meets its documented requirements through the testing pyramid:

- Unit tests confirmed the correctness of authentication utilities, order calculations, and stock validation logic.
- Integration tests proved that API endpoints behave correctly: successful and failed signups/logins, product CRUD operations, order placement and status updates, and admin-only access.
- End-to-end tests demonstrated that the key user journeys (customer ordering, admin product/ order management) function without errors across the full stack.

All tests pass, and the system's behaviour matches its specifications.

Validation (Does the software solve the right problem?)
We validated the system against real user needs discovered during persona analysis and edge-case exploration:

- Customers can browse, search, and purchase campus products – solving the problem of centralising university merchandise access.

- Administrators can manage inventory and orders through a dedicated dashboard, eliminating
  manual stock tracking.
- Edge cases (stock zero, duplicate order prevention, admin delete protection, auto-cancel) prevent user frustration and data integrity issues, matching the expectations of a professional
  e-commerce system.
- Informal user testing confirmed the interface is intuitive and the ordering process feels fast and reliable.

Therefore, the software not only works correctly but also addresses the intended problem for both
customer and admin actors.

Appendix A – Backend Test Files (pytest)
All unit and integration tests are located in backend/tests/.

- conftest.py sets up a temporary SQLite database and TestClient.
- unit/ contains test_auth_utils.py and test_order_logic.py.
- integration/ contains test_auth.py, test_admin_auth.py, test_products.py, test_orders.py.

Run them with: pytest tests/ -v

Appendix B – Playwright Test Structure

- playwright.config.js configured for baseURL http://localhost:3000.
- tests/pages/ folder holds the Page Object Model classes.
- tests/e2e/ folder holds the four .spec.js files.
- tests/fixtures/ contains a sample product image for upload tests.